DEVELOPER CONTEST SUBMISSION

Spark Repair Estimator

Mobile PWA for On-Site Repair Cost Estimation

Submitted by: Khush Modi

Date: June 30, 2026

Repository: https://github.com/Khushnew/spark-estimator/

1. Most Interesting Design Decision

The biggest UX decision was implementing **in-place total updates** instead of full re-renders when users change quantities. In the reference app, every keystroke triggers a complete DOM rebuild, causing input focus loss and visual flicker. I solved this by mutating state and surgically updating only the affected line total, group total, tab badge, header grand total, and progress bar. This keeps the quantity input focused while numbers animate smoothly around it, making the app feel native rather than web-based.

Second, I expanded the data model beyond the reference implementation to support **7 room types** (adding Bedroom and Living/Common Areas as first-class multi-instance rooms), while keeping the UI intuitive. Each room type gets its own tab with independently scrollable sub-tabs for instances, so agents can navigate a 3-bedroom, 2-bathroom house without cognitive overload.

2. What Is Broken or Fragile

Known Limitation **Photo Storage**

Photos stored as base64 data URLs in localStorage are subject to browser quota limits (~5-10MB). Large photo collections on older devices may hit the quota ceiling.

Known Limitation **OCR Accuracy**

Tesseract.js serial number parsing works best on well-lit, straight-on photos. Glare, blur, or angled shots reduce accuracy significantly.

Fragile **CDN Dependencies**

The app loads Tailwind, XLSX, JSZip, and Tesseract from CDNs. If a user has no internet and the resources are not cached, export and OCR features fail.

Fragile **Single HTML File Size**

At ~2,000 lines of inline JS/CSS, the file is large. A syntax error anywhere could break the entire app since there are no module boundaries.

3. Creative Addition: Deal Analyzer

I built a **Deal Analyzer** that calculates profit margin on the spot. Agents enter the After Repair Value (ARV), purchase price, and estimated holding weeks. The tool computes total investment, estimated profit, and margin percentage, color-coding the result (green for above-target margin, yellow for marginal, red for a loss). This directly addresses Spark Homes' core business question: not just "what will repairs cost?" but "will this deal make money?"

> **Why this feature:** The brief describes a three-phase business (Acquisition, Renovation, Resale), but the reference app only addresses phase two. The Deal Analyzer bridges the gap, letting agents evaluate acquisition viability during the same walkthrough where they estimate repairs. It uses the live repair total as an input, so the numbers are always synchronized.

4. What I Would Ship Next (With 2 More Days)

- **Offline CDN bundling:** Inline all CDN resources so the app works fully offline without prior cache warming. This is the highest-impact reliability fix.
- **Cost history / project comparison:** A dashboard showing cost trends across projects (e.g., "average bathroom cost on last 10 flips was $4,200"), helping agents benchmark estimates.
- **IndexedDB for photo storage:** Replace localStorage with IndexedDB to eliminate the photo quota problem and support unlimited photo attachments.
- **Cloud sync option:** Optional backend for team-wide project sharing, so multiple agents can collaborate on the same estimate.

Role of AI Tools in Development

I used AI tools (Claude Code / Cursor) as an **accelerator, not a replacement**, for my own thinking. I designed the architecture (7 room types, in-place updates, deal analyzer integration), made all UX decisions, and set quality standards. AI helped with rapid implementation of the rendering engine, event delegation, Excel export formatting, and PWA boilerplate. I reviewed every line, refactored for cleanliness, and made judgment calls on trade-offs. The final code represents a collaboration where I provided direction and AI provided velocity.

Built with vanilla JavaScript, Tailwind CSS, and passion for shipping products that feel fast and reliable in the field. Optimized for a phone in a real house, not for a screenshot.